Programowanie zaawansowane · laboratorium 4

Apache POI i Apache Commons CSV

Odczyt i zapis plików Excel i CSV w Javie

Igor Krzystek, Filip Kontek

	A	B	C
1	Imie	Stanowisko	Pensja
2	Jan Kowalski	Programista	15000
3	Anna Nowak	Tester	10000
4	Filip Kontek	Student	6
5	Igor Krzystek	Koszykarz	7

Pracownicy +

Te same dane, dwa różne formaty

pracownicy.csv

```
Imie;Stanowisko;Pensja;Adres
Jan Kowalski;Programista;15000,50;"ul. Długa 5; m. 3"
Anna Nowak;Tester;10000,00;Kraków
```

pracownicy.xlsx

	A	B	C	D
1	Imie	Stanowisko	Pensja	Adres
2	Jan Kowalski	Programista	15 000,50 zł	ul. Długa 5; m. 3
3	Anna Nowak	Tester	10 000,00 zł	Kraków

	CSV	**Excel** (**.xlsx**)
Zawartość	zwykły tekst, jedna tabela	archiwum ZIP z XML, wiele arkuszy
Typy danych	brak, wszystko to String	liczby, daty, wartości logiczne, formuły
Wygląd	brak	style, formaty liczb, kolory
Pamięć	czytamy wiersz po wierszu	XSSF trzyma cały plik w pamięci
Problemy	separator, kodowanie, cudzysłowy	daty jako liczby, formuły, rozmiar

W obu przypadkach program potrzebuje tego samego:

```
record Pracownik(String imie, String stanowisko, BigDecimal pensja) {}
```

Obie biblioteki dodajemy z Mavena

```
<properties>
    <poi.version>5.4.0</poi.version>
    <commons-csv.version>1.10.0</commons-csv.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.apache.poi</groupId>
        <artifactId>poi-ooxml</artifactId>
        <version>${poi.version}</version>
    </dependency>
    <dependency>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-csv</artifactId>
        <version>${commons-csv.version}</version>
    </dependency>
</dependencies>
```

`poi-ooxml`

Pliki .xlsx (XSSF, SXSSF). Sam dociąga bazowe poi, xmlbeans i commons-compress. Samo poi nie wystarczy do nowego formatu .xlsx

`commons-csv`

CSVFormat, CSVParser, CSVRecord, CSVPrinter.

Odczyt CSV do listy obiektów

```
CSVFormat format = CSVFormat.DEFAULT.builder()
        .setDelimiter(';')
        .setHeader()              // nagłówki z 1. linii
        .setIgnoreEmptyLines(true)
        .build();
List<Pracownik> lista = new ArrayList<>();
try (Reader in = Files.newBufferedReader(
             Path.of("pracownicy.csv"), StandardCharsets.UTF_8);
     CSVParser parser = format.parse(in)) {
    for (CSVRecord r : parser) {
        lista.add(new Pracownik(
                r.get("Imie"),
                r.get("Stanowisko"),
                new BigDecimal(r.get("Pensja").replace(',', '.'))));
    }
}
```

`CSVFormat`

Opis pliku: separator, nagłówki, cytowanie.

`CSVParser`

Czyta plik. Jest Iterable<CSVRecord>, więc wystarczy pętla for.

`CSVRecord`

Jeden wiersz. get("Imie") po nazwie albo get(0) po indeksie.

`CSVPrinter`

Zapis, za chwilę.

CSVFormat – dialekt pliku

Gotowe formaty

Stała	Separator	Cecha
DEFAULT	,	pomija puste linie
RFC4180	,	ściśle według RFC 4180
EXCEL	,	dopuszcza puste nazwy kolumn
TDF	tabulator	obcina spacje wokół wartości

Polski Excel zapisuje CSV ze średnikiem, więc prawie zawsze zmienimy separator.

Najważniejsze opcje buildera

Metoda	Efekt
setDelimiter(';')	separator pól
setHeader()	nagłówki z 1. linii
setHeader("A", "B")	własne nagłówki
setSkipHeaderRecord(true)	pomiń 1. linię pliku
setTrim(true)	obetnij spacje
setNullString("")	puste pole jako null

CSVFormat jest niezmienny: builder() robi kopię. Format trzymamy jako stałą i używamy go do odczytu i zapisu.

Uszkodzone i błędne pliki

```
try (CSVParser parser = format.parse(in)) {
    for (String kol : List.of("Imie", "Stanowisko", "Pensja"))
        if (!parser.getHeaderMap().containsKey(kol))
            throw new IllegalStateException("Brak kolumny: " + kol);

    for (CSVRecord r : parser) {
        if (!r.isConsistent() || r.get("Pensja").isBlank())
            throw new IllegalStateException(
                    "Błędny rekord nr " + r.getRecordNumber());
        // mapowanie na Pracownik jak na poprzednim slajdzie
    }
}
```

Brak kolumny

getHeaderMap() przed pętlą, komunikat z nazwą kolumny.

Zła liczba pól

isConsistent() porównuje rekord z nagłówkiem.

Puste pole

isBlank(), potem błąd albo wartość domyślna.

Przecinek dziesiętny

replace(',', '.') albo NumberFormat dla polskiego Locale.

Sanity check: przy dużym pliku warto wczytać najpierw kilka rekordów i sprawdzić poprawność struktury.
Wielkość liter: dla parsera ma znaczenie.
Split vs Parser: split(„;”) rozbija przy każdym „;” a parser czyta tekst w cudzysłowach jako jedno pole

Zapis CSV (cudzysłowy dodaje CSVPrinter)

```
CSVFormat out = CSVFormat.DEFAULT.builder()
        .setDelimiter(';')
        .setHeader("Imie", "Stanowisko", "Pensja")
        .build();

try (Writer w = Files.newBufferedWriter(
             Path.of("wynik.csv"), StandardCharsets.UTF_8)) {
    w.write('');        // BOM, przed nagłówkiem
    try (CSVPrinter printer = new CSVPrinter(w, out)) {
        for (Pracownik p : lista)
            printer.printRecord(p.imie(), p.stanowisko(),
                    p.pensja());
    }
}
```

wynik.csv

```
Imie;Stanowisko;Pensja
Jan Kowalski;Programista;15000.50
"Nowak; Anna";Tester;10000.00
```

Pole ze średnikiem trafiło w cudzysłów bez naszego udziału.

BOM piszemy przed utworzeniem CSVPrinter, bo ten wypisuje nagłówek już w konstruktorze.

BigDecimal zapisuje się z kropką. Polski Excel może wziąć to za tekst.

POI – odczyt .xlsx do listy rekordów

```
record Person(String name, LocalDate birthDate, BigDecimal salary) {}

DataFormatter fmt = new DataFormatter();
List<Person> osoby = new ArrayList<>();
try (Workbook wb = WorkbookFactory.create(new File("osoby.xlsx"))) {
    Sheet sheet = wb.getSheetAt(0);
    for (Row row : sheet) {
        if (row.getRowNum() == 0) continue;      // nagłówek
        String name = fmt.formatCellValue(row.getCell(0));
        LocalDate birth = row.getCell(1)
                .getLocalDateTimeCellValue().toLocalDate();
        BigDecimal salary = BigDecimal.valueOf(
                row.getCell(2).getNumericCellValue());
        osoby.add(new Person(name, birth, salary));
    }
}
```

`WorkbookFactory`

Sam rozpozna .xls i .xlsx.

`for (Row row : sheet)`

Pomija wiersze, których w pliku nie ma.

`DataFormatter`

Tekst taki, jak w Excelu. Nie rzuca wyjątku dla liczby ani dla null.

Pusta komórka

getCell() zwraca null. Pomaga getCell(i, CREATE_NULL_AS_BLANK).

Numeracja od 0 – różnica między excelem a java

Daty to liczby, formuły to przepis

Daty jako liczby:

Wywołanie	Wynik
getNumericCellValue()	45231.0
getLocalDateTimeCellValue()	2023-11-01T00:00
fmt.formatCellValue(cell)	01.11.2023
DateUtil.isCellDateFormatted(cell)	true
getStringCellValue()	IllegalStateException

Excel trzyma datę jako liczbę dni od początku 1900 roku. Typ daty jest tylko w formacie komórki.

Przeliczanie formuł:

getCellType() zwraca FORMULA, a getNumericCellValue() wynik zapamiętany przy ostatnim zapisie pliku. Po zmianie danych w Javie liczymy od nowa:

```
FormulaEvaluator ev = wb
        .getCreationHelper()
        .createFormulaEvaluator();
double roczna = ev.evaluate(cell)
        .getNumberValue();
```

Tekst z wynikiem formuły: fmt.formatCellValue(cell, ev)

Zapis .xlsx: style tworzymy raz

```
try (Workbook wb = new XSSFWorkbook();
     OutputStream out = Files.newOutputStream(Path.of("raport.xlsx"))) {
    Sheet sheet = wb.createSheet("Pracownicy");
    CellStyle kwota = wb.createCellStyle();       // raz, przed pętlą
    kwota.setDataFormat(
            wb.createDataFormat().getFormat("#,##0.00 \"zł\""));
    int r = 0;
    for (Person p : osoby) {
        Row row = sheet.createRow(r++);
        row.createCell(0).setCellValue(p.name());
        Cell c = row.createCell(1);
        c.setCellValue(p.salary().doubleValue());
        c.setCellStyle(kwota);
    }
    sheet.autoSizeColumn(0);
    wb.write(out);
}
```

Styl należy do pliku

Nie należy używać createCellStyle() w pętli - to nowy styl dla każdej komórki.
Limit stylów dla .xlsx: 64 000.

Pieniądze

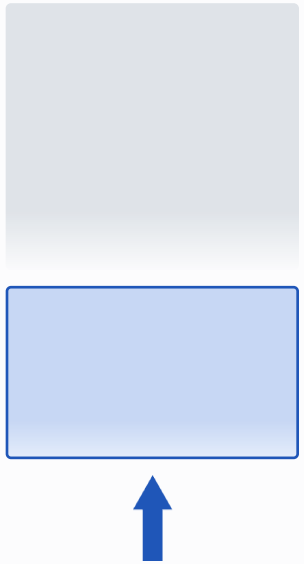
W kodzie BigDecimal. W komórce liczba z formatem kwoty albo tekst, gdy liczy się każdy grosz.

`0.1 + 0.2 = 0.30000000000000004`

Daty

Bez stylu z formatem daty komórka pokaże 45231 – surową liczbę bazową.

Duże pliki: SXSSF trzyma w pamięci tylko okno



Starsze wiersze: zapisane do pliku tymczasowego na dysku. Nie da się do nich wrócić.

Okno: ostatnie 100 wierszy w pamięci.

`sheet.createRow(i)`

```
try (SXSSFWorkbook wb = new SXSSFWorkbook(100)) {
    Sheet sheet = wb.createSheet("Dane");
    for (int i = 0; i < 1_000_000; i++) {
        Row row = sheet.createRow(i);
        row.createCell(0).setCellValue(i);
    }
    try (OutputStream out = Files.newOutputStream(plik)) {
        wb.write(out);
    }
    wb.dispose();        // usuwa pliki tymczasowe
}
```

XSSF trzyma każdą komórkę jako obiekt. Setki tysięcy wierszy kończą się OutOfMemoryError.

Limit arkusza to 1 048 576 wierszy. Większe dane dzielimy na kilka arkuszy.

Odczyt wielkich plików to osobne API zdarzeniowe (XSSFReader).

Demo: konwerter CSV ↠ Excel

dane.csv

```
Imie;Stanowisko;Pensja
Jan Kowalski;Programista;15000
Anna Nowak;Tester;10000
Filip Kontek;Student;6
Igor Krzystek;Koszykarz;7
```



wynik.xlsx · arkusz Pracownicy

	A	B	C
1	Jan Kowalski	Programista	15000
2	Anna Nowak	Tester	10000
3	Filip Kontek	Student	6
4	Igor Krzystek	Koszykarz	7

```
TU WRZUC JAKIS KLUCZOWY KAWALEK/KAWALKI KODU Z KONWERTERA JAK BĘDZIE DZIALAC NA PEWNO DORBZE EZ
```

Dobre praktyki

CSV

- Separator, nagłówki i cytowanie w jednej stałej CSVFormat
- Zawsze jawne UTF-8, BOM gdy plik otworzy Excel na Windowsie
- Cytowanie zostawiamy bibliotece: żadnego split ani String.join
- Duże pliki iteracyjnie, bez zbierania wszystkiego w listę
- Walidacja nagłówków i sanity check na starcie

Excel / POI

- Daty przez getLocalDateTimeCellValue() albo DataFormatter
- Formuły przez FormulaEvaluator
- Style tworzone raz na cały skoroszyt
- Kwoty w BigDecimal, nigdy w double
- Powyżej ok. 100 tys. wierszy SXSSF i dispose()

Wspólne: każdy plik w try-with-resources, mapowanie na rekordy w jednym miejscu.

Dziękujemy za uwagę

Dokumentacja

poi.apache.org/components/spreadsheet

commons.apache.org/proper/commons-csv